ENSEEIHT

2A-SN-APP
Sciences des Réseaux et IA

Rapport de projet

Clustering de terminaux utilisateurs dans un réseau de constellations de satellites

---

Sujet : 2

Année universitaire : 2025–2026

Encadrants :
Riadh DHAOU
Ali BOURIGA

Membres du groupe

Emilien FIEU
Hamza CHOUCHENNE
Nassim KHOUJANE
Denisa RIPICEANU
Sarah DAMBIELLE
Ismail AL RIYAMI

Table des matières

1 Introduction **2**

2 Étude bibliographique **2**

2.1 Données fournies . . . 2

2.2 Présentation des anciens travaux d'étudiants . . . 2

2.3 Présentation des algorithmes existants . . . 3

2.3.1 Algorithme GeoClust[1] . . . 3

2.3.2 BK-Means et TGBP de Chien et al.[2] . . . 3

2.3.3 Approche p-center de Tang et al.[3] . . . 4

3 Définition de l'étude **5**

3.1 Pré-traitements . . . 5

3.1.1 Indexation Spatiale (K-D Tree) . . . 5

3.1.2 Tri des utilisateurs par demande décroissante . . . 5

3.2 Traitement . . . 6

3.2.1 Algorithme p-center optimisé . . . 6

3.2.2 Algorithme Gravity Mean-Shift . . . 8

3.2.3 Approche top-down avec fusion locale par slack . . . 9

3.3 Post-traitements . . . 10

3.3.1 Le pruning . . . 10

3.3.2 Fusion de clusters . . . 11

3.3.3 Boyle-Dykstra . . . 11

4 Résultats obtenus **12**

4.1 Algorithme p-center optimisé . . . 12

4.1.1 Résultats chiffrés . . . 12

4.1.2 Complexité . . . 12

4.2 Algorithme Gravity Mean-Shift . . . 13

4.2.1 Résultats chiffrés . . . 13

4.2.2 Complexité . . . 13

4.3 Algorithme split-merge avec réduction de Boyle-Dykstra . . . 14

4.3.1 Résultats chiffrés . . . 14

4.3.2 Complexité . . . 14

4.4 Comparaison globale des approches . . . 14

5 Conclusion **16**

A Annexes **17**

A.1 Graphiques complémentaires de l'algorithme p-center optimisé . . . 17

A.1.1 Comparaison globale entre CIR et PIR . . . 17

A.1.2 Distribution des rayons et des taux de remplissage . . . 17

A.1.3 Répartition géographique des terminaux et des centres . . . 18

1 Introduction

Le déploiement des constellations de satellites en orbite basse (LEO) permet aujourd'hui de fournir un accès internet à haut débit sur l'ensemble du globe. Pour communiquer avec les terminaux terrestres, ces satellites orientent dynamiquement leurs antennes pour former des faisceaux qui éclairent des zones géographiques spécifiques.

L'objectif de ce projet est de concevoir un algorithme capable de regrouper efficacement près de 90 000 utilisateurs terrestres en différents groupes, appelés clusters. Chaque cluster défini correspondra à la cible d'un de ces faisceaux satellites.

Pour que la solution soit réaliste et applicable, elle doit impérativement respecter deux contraintes physiques. La première est géométrique : l'empreinte au sol d'un faisceau a un diamètre fixe de 90 km. Aucun utilisateur ne peut donc se trouver à plus de 45 km du centre de son cluster. La seconde contrainte est liée à la capacité : la somme des débits demandés par les terminaux d'un même cluster ne doit pas dépasser la limite de bande passante du satellite. Nous testerons notre approche sur trois scénarios imposant un plafond strict de 1, 2 et 4 Gbps par faisceau.

Notre but principal est de minimiser le nombre total de clusters créés. L'intérêt de cette minimisation est direct : moins il y a de zones différentes à couvrir, moins le satellite perd de temps à basculer d'une zone à l'autre. Cela permet de réduire les files d'attente des paquets de données et d'améliorer la latence du réseau.

Dans ce rapport, nous commencerons par analyser plusieurs approches issues de la littérature concernant le placement de faisceaux. Nous expliquerons pourquoi les algorithmes classiques de clustering montrent très vite leurs limites mathématiques face à un jeu de données aussi massif. Enfin, nous présenterons les algorithmes, les optimisations de pré-traitement et les étapes de fusion que nous avons développés pour résoudre ce problème.

2 Étude bibliographique

2.1 Données fournies

Nous avons à notre disposition, un fichier CSV contenant la liste des utilisateurs avec leur position géographique (longitude, latitude), leur PIR [1] ainsi que le CIR [2], ainsi que la classe de service de l'utilisateur, 0, 1 ou 2.

2.2 Présentation des anciens travaux d'étudiants

Les trois stratégies de clustering étudiées :

— **Méthode 1 (Parcours simple) :** Intégration de chaque utilisateur dans un cluster existant si la zone géographique et la bande passante le permettent. Dans le cas contraire, un nouveau cluster est créé autour de lui.

— **Méthode 2 (Tri par QoS) :** Tri initial des utilisateurs selon leur niveau de service (PIR). L'algorithme remplit au maximum un cluster autour du premier utilisateur de la liste, puis répète l'opération avec les utilisateurs restants.

1. Peak Information Rate : débit maximum que l'utilisateur peut atteindre dans un cas idéal
2. Committed Information Rate : débit minimum garanti par le réseau à l'utilisateur

— **Méthode 3 (Capacité maximale) :** Création d'un cluster potentiel pour chaque utilisateur, puis tri de ces clusters par capacité d'accueil décroissante. Un cluster est validé uniquement si *tous* ses utilisateurs sont encore disponibles. Les derniers utilisateurs isolés reçoivent un cluster individuel.

Optimisation et résultats comparatifs :

— **Post-traitement de fusion (« 2 in 1 ») :** Pour chaque méthode, un algorithme effectue plusieurs passes finales pour fusionner les clusters entre eux, tant que les contraintes globales restent respectées.

— **Bilan des Méthodes 1 et 2 :** Ces approches génèrent initialement entre 20 935 et 21 107 clusters. Après l'application du post-traitement, ce nombre chute entre 16 284 et 16 533.

— **Bilan de la Méthode 3 :** En associant cette méthode de sélection par capacité à la fusion finale, les étudiants ont obtenu le résultat le plus optimisé de leur étude, avec un minimum de 16 193 clusters.

2.3 Présentation des algorithmes existants

Plusieurs travaux récents abordent le problème du placement de faisceaux dans les constellations LEO en utilisant des techniques de clustering.

2.3.1 Algorithme GeoClust[1]

Cet algorithme a pour but d'optimiser le clustering des faisceaux de satellites. L'objectif est de réduire le nombre de clusters tout en garantissant un débit maximal pour les utilisateurs.

Cette approche s'effectue en deux étapes :

— **1re étape :** Un algorithme glouton qui assure que tous les utilisateurs sont affectés à un cluster.

— **2e étape (optimisation-Boyle-Dykstra) :** À chaque itération, les utilisateurs sont réaffectés au centre le plus proche, puis les centres sont repositionnés afin d'améliorer le regroupement sous la contrainte stricte de ne jamais dépasser le rayon maximal de couverture autorisé pour un faisceau satellite.

Cette approche présente une complexité qui se calcule en deux phases. La phase d'initialisation gloutonne présente une complexité de $O(N^2)$. Elle est suivie d'une phase d'optimisation des centres (Boyle-Dykstra) d'une complexité d'environ $O(I \times N \times K)$. La complexité temporelle globale s'approche par conséquent de $O(N^2 + I \times N \times K)$, où N représente le nombre total d'utilisateurs terrestres à répartir, K désigne le nombre de clusters finaux générés et I correspond au nombre d'itérations nécessaires avant la convergence de l'algorithme. Cependant, pour simplifier, puisque le terme N^2 est dominant, on peut considérer que la complexité globale est tout simplement de $O(N^2)$.

Cette méthode peut en partie s'appliquer à notre cas d'étude. En effet, leur approche tente de limiter la taille géographique des clusters, mais elle ne prend pas en compte notre limitation sur le débit maximal par cluster.

2.3.2 BK-Means et TGBP de Chien et al.[2]

À l'inverse de GeoClust, Chien et al. proposent deux autres algorithmes aux approches radicalement différentes pour tenter de résoudre le même problème.

La première, le **Bisection K-Means (BK-Means)**, cherche le nombre optimal de faisceaux en combinant le K-means classique avec une recherche par dichotomie. Au lieu de tester le nombre de clusters un par un,

l'algorithme évalue la valeur médiane d'un intervalle et vérifie si la contrainte géométrique est respectée pour chaque groupe. Si la configuration est valide, on resserre l'intervalle vers le bas pour minimiser les clusters ; sinon, on cherche plus haut.

Dans le pire des cas, la complexité du BK-Means est de $O(N^3 \cdot I \cdot \log N)$, avec N le nombre d'utilisateurs et I le nombre d'itérations. Nous avons écarté cette méthode car pour nos 90 000 utilisateurs, le terme en N^3 fait exploser les calculs. Rien que pour la première étape de la dichotomie, cela demande plusieurs milliards d'opérations. Ces temps de calcul immenses rendent cette méthode classique inutilisable en pratique pour notre projet.

Face aux limites temporelles de cette première méthode, Chien et al. proposent une seconde approche : le **TGBP (Two-phase Greedy Beam Placement)**. Cet algorithme basé sur la coloration de graphe fonctionne en deux étapes. D'abord, lors de la coloration, chaque utilisateur devient un nœud du graphe. On relie par une arête les utilisateurs trop éloignés pour partager un même faisceau, c'est-à-dire avec une distance supérieure à 90 km. Un algorithme glouton colore ensuite ces nœuds en commençant par ceux qui ont le plus de connexions. Chaque couleur représente un faisceau, ce qui détermine le nombre total de faisceaux nécessaires. Ensuite, lors de l'équilibrage, l'algorithme déplace certains utilisateurs des faisceaux surchargés vers des faisceaux plus vides, tout en respectant la contrainte de rayon.

Sa complexité est de $O(N^3K^2)$. La coloration compare les N utilisateurs sur les K faisceaux, et la réaffectation teste le transfert de chaque utilisateur en vérifiant la distance avec les membres du faisceau cible. En pratique, cela reste rapide car le nombre de faisceaux K est très petit par rapport au nombre d'utilisateurs N.

Même si le TGBP est efficace pour équilibrer la charge, sa logique n'est pas idéale pour notre sujet. Notre but principal est de minimiser strictement le nombre de clusters. Or, la répartition de charge du TGBP bloque la structure des faisceaux au lieu de chercher à les fusionner. Des algorithmes avec un meilleur post-traitement de fusion sont donc plus adaptés pour notre projet.

2.3.3 Approche p-center de Tang et al.[3]

L'algorithme p-center proposé par Tang et al. s'intéresse aux satellites fonctionnant en *beam-hopping* (saut de faisceaux). Les auteurs prouvent mathématiquement que moins il y a de faisceaux, plus le délai d'attente du réseau est court, ce qui justifie directement notre propre objectif de minimiser le nombre de clusters.

Leur méthode de résolution est itérative. Le fonctionnement est le suivant : on commence par tester si un seul cluster ($p = 1$) suffit à couvrir toute la zone. Pour cela, les utilisateurs sont assignés au centre le plus proche, puis on recalcule ce centre de manière à réduire son rayon au maximum. Si la contrainte de distance (un rayon de 45 km) n'est pas respectée, on ajoute un cluster supplémentaire ($p = 2, 3 \ldots$) et on répète l'opération jusqu'à obtenir une configuration géométrique valide pour tous les terminaux.

La complexité temporelle de cet algorithme est de $O(N^3)$, où N représente le nombre d'utilisateurs. Sur notre fichier de 90 000 lignes, cette complexité cubique entraînerait des temps de calcul trop longs (doubler le nombre d'utilisateurs multiplierait le temps de calcul par huit). Cependant, les auteurs précisent qu'en simplifiant l'algorithme, notamment en supprimant certaines boucles de réaffectation des utilisateurs, cette complexité peut être ramenée à $O(N^2)$.

Le modèle d'origine possède quelques limites pour notre cas d'étude : il est purement géométrique (il ignore totalement la capacité en Gbps) et il interdit la superposition des clusters. Malgré cela, son

principe d'exploration itérative est excellent, car il garantit de trouver le plus petit nombre de clusters valide. C'est pourquoi nous avons décidé de retenir cette stratégie p-center comme l'une de nos approches principales. Nous l'adapterons par la suite pour y intégrer nos limites strictes de capacité et nos méthodes d'optimisation temporelles.

3 Définition de l'étude

3.1 Pré-traitements

Avant de lancer le processus d'optimisation, quel que soit l'algorithme de clustering choisi (P-Center, Gravity Mean-Shift ou autre), nous avons mis en place une phase universelle de pré-traitement des données. L'objectif de ce socle commun est double : réduire drastiquement les temps de calcul globaux et assurer une base facilitant le respect des contraintes de capacité et de distance.

3.1.1 Indexation Spatiale (K-D Tree)

La première contrainte de ce projet est la taille du jeu de données : nous devons traiter environ 90 000 utilisateurs. Un utilisateur ne peut se connecter qu'à un satellite situé à moins de 45 km.

Si nos algorithmes devaient vérifier la distance entre chaque nouvel emplacement de satellite et l'intégralité des 90 000 utilisateurs à chaque fois, le temps d'exécution serait beaucoup trop long (complexité quadratique). Pour optimiser cela, nous avons utilisé une structure de données appelée K-D Tree (implémentée avec le module cKDTree de Python[4]).

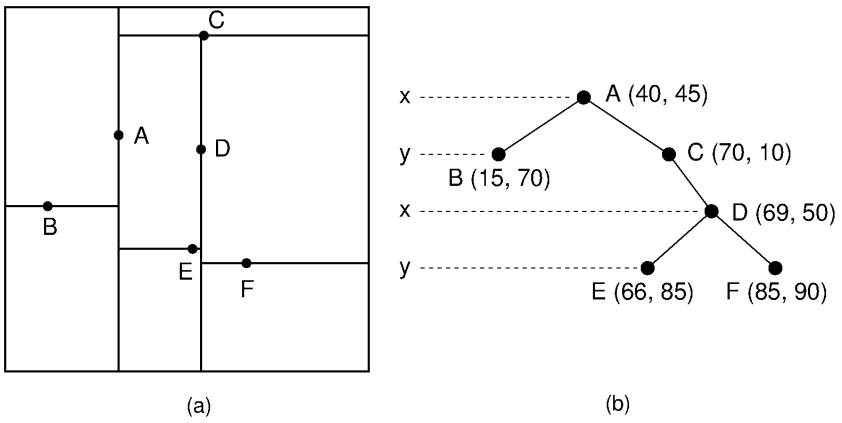


FIGURE 1 – Exemple d'un arbre k-d[5]. (a) La décomposition par arbre k-d pour une région de 128×128 unités contenant sept points de données. (b) L'arbre k-d correspondant à la région (a).

Le K-D Tree va pré-organiser les points géographiques sous forme d'arbre. Lorsqu'un algorithme a besoin de connaître les utilisateurs situés à moins de 45 km d'un point donné, l'arbre permet de cibler directement la bonne zone géographique sans vérifier le reste de la carte. Cela accélère énormément les recherches de voisinage et permet à nos algorithmes de tourner en quelques secondes.

3.1.2 Tri des utilisateurs par demande décroissante

Le second enjeu concerne la contrainte de capacité des satellites (1, 2 ou 4 Gbps). Ce cas d'usage ressemble au problème classique du Bin Packing. Si un algorithme place d'abord une multitude de petits

utilisateurs dans un satellite, il risque de ne plus avoir assez de place pour accueillir un utilisateur avec une forte demande par la suite. Cela forcerait l'algorithme à créer un nouveau satellite presque vide, ce qui n'est pas optimal.

Pour éviter ce problème, nous opérons un tri initial de tous les utilisateurs : nous les classons par ordre décroissant de leur demande en bande passante.

Grâce à ce tri simple, les algorithmes vont traiter en priorité les utilisateurs les plus "gourmands" pour les placer au centre des clusters. Les nombreux profils ayant une demande plus faible viendront ensuite combler très précisément la capacité restante de chaque satellite. C'est une méthode très simple à implémenter, mais qui aide tous nos modèles à minimiser considérablement le nombre final de satellites déployés.

3.2 Traitement

3.2.1 Algorithme p-center optimisé

Principe général du p-center optimisé

Notre approche s'inspire du problème du *p-center*, qui consiste à couvrir un ensemble d'utilisateurs avec des centres en limitant la distance maximale entre un utilisateur et le centre auquel il est affecté. Ici, chaque cluster doit respecter un rayon maximal de 45 km.

Notre méthode ne résout pas exactement le p-center classique. Elle correspond plutôt à une heuristique gloutonne pour une variante capacitaire du p-center. Un cluster est accepté uniquement s'il respecte simultanément la contrainte géographique et la contrainte de capacité : $rayon(cluster) \leq 45\ km$, $debit(cluster) \leq C$ où $C \in \{1, 2, 4\}$ Gbps selon le scénario étudié.

L'algorithme est exécuté avec deux définitions de la demande : le CIR, qui correspond au débit garanti, et le PIR, qui correspond au débit maximal. Cette comparaison permet d'évaluer l'impact du choix de la demande sur le nombre de clusters nécessaires.

Phase de pré-traitement

La phase de pré-traitement consiste d'abord à charger les terminaux avec leurs coordonnées géographiques et leur demande en débit. Selon l'expérience réalisée, la demande utilisée est soit le CIR, soit le PIR, converti en Gbps afin d'être comparé directement aux capacités des clusters.

Pour rendre tous les scénarios faisables, les terminaux dont la demande dépasse la capacité C sont découpés en terminaux virtuels situés au même endroit. Cette étape intervient notamment pour le scénario PIR à 1 Gbps, où certains terminaux demandent plus que la capacité autorisée. La demande totale est ainsi conservée.

Les terminaux sont ensuite regroupés en micro-zones géographiques. Ces micro-zones servent uniquement à guider l'ordre de construction des clusters, en traitant d'abord les zones les plus chargées. La validation finale reste effectuée sur les terminaux eux-mêmes. Les recherches de voisinage sont accélérées à l'aide d'un KD-tree, comme présenté dans la partie pré-traitement.

Algorithme

L'algorithme construit les clusters de manière gloutonne. À chaque étape, il sélectionne une micro-zone encore non couverte, choisit un terminal graine, puis recherche les candidats dans un rayon de 90 km. Ce

rayon est supérieur au rayon maximal du cluster, car deux terminaux distants d'environ 90 km peuvent être couverts par un même centre placé entre eux.

Les candidats sont évalués par un score combinant distance, demande, capacité restante et appartenance à la micro-zone courante. Un candidat est ajouté uniquement si le débit total reste inférieur à C et si le cercle minimum englobant des membres du cluster conserve un rayon inférieur ou égal à 45 km.

Algorithm 1 Heuristique p-center capacitaire optimisée

for chaque demande $D \in \{CIR, PIR\}$ **do**
  for chaque capacité $C \in \{1, 2, 4\}$ Gbps **do**
    Convertir la demande D en Gbps
    Créer les terminaux virtuels si une demande dépasse C
    Construire les micro-zones
    Construire un KD-tree sur les positions géographiques
    while il reste des terminaux non couverts **do**
      Choisir une micro-zone dense non entièrement couverte
      Choisir un terminal graine dans cette zone
      Rechercher les candidats dans un rayon de 90 km
      Initialiser un cluster avec le terminal graine
      while le cluster peut encore être rempli **do**
        Classer les candidats selon le score glouton
        for chaque candidat parmi les meilleurs candidats **do**
          Calculer le débit après ajout
          Calculer le cercle minimum englobant après ajout
          if $debit \leq C$ et $rayon \leq 45$ km **then**
            Ajouter le candidat au cluster
          end if
        end for
      end while
      Marquer les terminaux du cluster comme couverts
    end while
  end for
end for

La superposition partielle ou totale des clusters est autorisée : plusieurs clusters peuvent couvrir une même zone si la demande locale dépasse la capacité d'un seul cluster. Une tentative de fusion locale a été testée en post-traitement, mais elle n'a pas réduit le nombre de clusters dans la version finale. Cela indique que les clusters produits sont déjà localement saturés par les contraintes de rayon et de capacité.

Vérification

Après la construction, plusieurs vérifications automatiques sont effectuées. On contrôle que chaque terminal virtuel est affecté exactement une fois, que le débit total est conservé, et que chaque cluster respecte bien : $rayon(cluster) \leq 45\ km$, $debit(cluster) \leq C$.

Les résultats sont également comparés à la borne inférieure par débit : $borne_debit = \lceil \frac{debit_total}{C} \rceil$

Cette borne est optimiste car elle ignore la géographie, mais elle permet de vérifier la cohérence des résultats obtenus.

Complexité

La construction du KD-tree coûte $O(N \log N)$, où N est le nombre de terminaux virtuels. Pour chaque cluster, l'algorithme effectue ensuite une recherche locale de voisins. Dans le pire cas, cette recherche peut retourner jusqu'à $O(N)$ candidats et être répétée pour au plus N clusters.

Le nombre de candidats réellement testés à chaque étape est borné par un paramètre q fixé dans l'implémentation. De plus, la taille maximale d'un cluster, notée $s_{\max}$, est limitée par la capacité du scénario. La complexité peut donc être majorée par $O(N^2 \log N + Nqs_{\max}^2)$.

Comme q est fixé et que $s_{\max}$ reste limité par la capacité, la complexité dominante est de l'ordre de $O(N^2 \log N)$ dans le pire cas. En pratique, le KD-tree, les micro-zones et la limitation du nombre de candidats testés rendent l'algorithme beaucoup plus efficace qu'une résolution exacte du p-center.

3.2.2 Algorithme Gravity Mean-Shift

En nous inspirant des modèles de Machine Learning non supervisés comme le Mean-Shift[6], nous avons conçu un algorithme de clustering pour répondre aux contraintes du projet.

Plutôt que de figer un satellite directement sur le premier utilisateur trouvé, notre algorithme simule un phénomène de gravité probabiliste. Lorsqu'un satellite est déployé, il calcule la position de tous les voisins non connectés dans un rayon de 45 km, puis il glisse littéralement sur la carte vers le centre de gravité de leur demande en bande passante. Ce déplacement itératif permet au satellite de se recentrer exactement sur les zones de plus forte densité, les "hotspots" du réseau.

Une fois la position optimale atteinte généralement en 3 itérations, le satellite se fixe. Il procède alors au remplissage de sa capacité en absorbant les utilisateurs environnants selon un algorithme glouton (tri par ordre décroissant de demande, cf. Pré-traitements), s'assurant ainsi d'optimiser chaque Gb disponible.

Algorithm 2 Heuristique Gravity Mean-Shift

```
Trier tous les utilisateurs par ordre décroissant de demande
for chaque utilisateur u non couvert do
    Initialiser le centre du satellite aux coordonnées de u
    // Première phase, déplacement vers le centre de gravité local
    for itération de 1 à 3 do
        Rechercher les utilisateurs non couverts dans un rayon de 45 km
        if aucun utilisateur n'est trouvé then
            Sortir de la boucle de déplacement
        end if
        Mettre à jour le centre sur le barycentre pondéré par les demandes
    end for
    // Deuxième phase, Bin Packing
    Créer un nouveau cluster au centre final trouvé
    Rechercher les candidats non couverts dans un rayon de 45 km du cluster
    Trier ces candidats par ordre décroissant de demande
    for chaque candidat v do
        if capacité actuelle du cluster + demande de v ≤ C_max then
            Ajouter v au cluster et le marquer comme couvert
        end if
    end for
end for
// Dernière phase, rattrapage des utilisateurs isolés
for chaque utilisateur u restant non couvert do
    Créer un cluster individuel centré sur u
end for
```

Analyse de la complexité

La complexité temporelle de cet algorithme est fortement optimisée grâce aux pré-traitements. Le tri initial de l'ensemble des utilisateurs s'effectue en $O(N \log N)$, avec N le nombre total d'utilisateurs.

Dans la boucle principale, la création d'un satellite implique seulement des recherches spatiales locales. Grâce au K-D Tree, chaque recherche de voisins dans un rayon de 45 km voit sa complexité chuter à environ $O(\log N + k)$, où k est le nombre de voisins trouvés. Sachant que le déplacement du satellite (phase 1) est borné à un temps constant (3 itérations maximum) et que le tri local des candidats lors du remplissage (phase 2) s'opère sur un très petit sous-ensemble $k \ll N$, le coût par cluster créé reste minime.

Globalement, la complexité moyenne de l'algorithme est dominée par le tri initial et le requêtage de l'arbre, ce qui donne un temps d'exécution asymptotique de l'ordre de $O(N \log N)$. En pratique, cette efficacité permet de traiter les 90 000 utilisateurs du dataset mondial en quelques secondes seulement.

3.2.3 Approche top-down avec fusion locale par slack

Ce traitement repose sur une approche *top-down*. Au lieu de construire les clusters un par un, on part d'un cluster initial contenant tous les utilisateurs, puis on le divise progressivement jusqu'à obtenir uniquement des clusters valides. Cette stratégie est bien adaptée à notre problème, car elle permet de traiter directement les deux contraintes importantes : le rayon maximal et le débit maximal par cluster.

À chaque itération, on identifie les clusters qui ne respectent pas les contraintes. Chacun de ces clusters est alors séparé en deux sous-clusters à l'aide d'une heuristique de division. Le processus est répété tant

qu'il reste des clusters invalides. On obtient ainsi une solution faisable, mais souvent trop fragmentée, avec des clusters valides qui pourraient encore être regroupés.

On applique ensuite une phase de fusion locale. Pour chaque cluster, on cherche des voisins avec lesquels une fusion serait possible sans dépasser ni la contrainte de rayon ni la contrainte de débit. Les fusions candidates sont ordonnées selon un critère de *slack*, c'est-à-dire selon la marge restante après fusion. L'idée est de privilégier les unions qui restent sûres géométriquement tout en utilisant mieux la capacité disponible. Cette phase permet de réduire le nombre de clusters sans remettre en cause la faisabilité de la solution.

Algorithm 3 Approche top-down avec fusion locale par slack

Require: ensemble d'utilisateurs U
créer un cluster initial contenant tous les utilisateurs
while il existe au moins un cluster invalide **do**
    sélectionner un cluster invalide
    le diviser en deux sous-clusters selon une heuristique de séparation
    remplacer le cluster initial par ces deux nouveaux clusters
end while
while il existe des fusions compatibles **do**
    générer les fusions candidates entre clusters proches
    trier ces fusions selon le critère de slack
    appliquer les meilleures fusions qui respectent encore les contraintes
end while
return ensemble final de clusters

Complexité algorithmique

Cette approche se décompose en deux phases. La phase de division dépend du nombre d'itérations nécessaires pour rendre tous les clusters valides, ainsi que du coût de chaque séparation. La phase de fusion, quant à elle, pourrait être quadratique en nombre de clusters si l'on testait toutes les paires possibles. Dans notre implémentation, ce coût est réduit par une recherche locale des voisins, ce qui évite une exploration exhaustive. En pratique, la phase de division reste la plus coûteuse, tandis que la fusion locale agit comme un raffinement final plus ciblé.

3.3 Post-traitements

Bien que nos différentes heuristiques génèrent rapidement des solutions valides respectant les contraintes strictes de distance et de capacité, ces méthodes laissent souvent une marge d'optimisation. En effet, de nombreux satellites se retrouvent sous-exploités en fin d'exécution avec une fragmentation de la demande.

Pour réduire le coût de déploiement global et maximiser le taux de remplissage du réseau, nous appliquons un post-traitement organisé en trois mécanismes. Ces post-traitements sont agnostiques et s'appliquent universellement pour nettoyer et compresser les résultats de tous les algorithmes présentés précédemment.

3.3.1 Le pruning

La première étape de post-traitement élimine les satellites les moins remplis de la carte. L'algorithme isole les clusters ayant une très faible charge et tente, pour chacun de leurs utilisateurs, de trouver un

satellite voisin à moins de 45 km disposant de la capacité résiduelle nécessaire. Si tous les utilisateurs d'un petit cluster parviennent à être réaffectés à d'autres satellites existants, le petit satellite est détruit. Ce processus vide efficacement les zones fragmentées sans ajouter de nouveaux points de connexion.

3.3.2 Fusion de clusters

Cette étape s'occupe des zones où la densité le permet : la fusion. On cherche à repérer deux satellites voisins qui ne sont remplis qu'à moitié de leur capacité. Si la somme de leurs charges ne dépasse pas la capacité maximale autorisée, l'algorithme tente de les combiner. Il calcule alors le centre global de tous les utilisateurs des deux clusters. Si ce nouveau centre parvient à couvrir l'ensemble des utilisateurs dans la limite stricte de 45 km, les deux anciens satellites sont supprimés au profit de ce nouveau satellite central. Effectuée sur plusieurs passes, cette méthode permet d'absorber énormément de structures redondantes.

3.3.3 Boyle-Dykstra

Ce post-traitement vise à réduire encore le nombre de clusters après la phase principale de clustering. Le principe est de tenter de supprimer certains petits clusters en redistribuant leurs utilisateurs vers des clusters voisins, tout en respectant les contraintes de rayon maximal et de débit maximal.

Pour cela, on sélectionne d'abord quelques clusters candidats autour du cluster source. On construit ensuite une matrice de préférences qui favorise les clusters les plus proches. Cette matrice est projetée par une méthode de type Dykstra[7] afin d'obtenir une affectation souple compatible avec les capacités disponibles. Enfin, cette affectation sert uniquement de guide pour effectuer une réaffectation discrète réelle des utilisateurs.

Algorithm 4 Post-traitement Boyle-Dykstra

Require: ensemble de clusters C
for chaque petit cluster source s **do**
    sélectionner quelques clusters voisins candidats
    if leur capacité libre totale est insuffisante **then**
        passer au cluster suivant
    end if
    for chaque utilisateur u de s **do**
        conserver seulement les clusters pouvant accueillir u
        if aucun cluster n'est possible **then**
            abandonner la suppression de s
        end if
    end for
    construire une matrice de préférences favorisant les clusters proches
    projeter cette matrice avec Dykstra
    tenter une réaffectation discrète guidée par cette projection
    if tous les utilisateurs de s sont replacés sans violer les contraintes **then**
        supprimer s
    end if
end for

Complexité algorithmique

Le coût de ce post-traitement dépend surtout de la taille du cluster source et du nombre de clusters

candidats considérés. La projection de type Dykstra sur une matrice de taille $s \times k$, où s est le nombre d'utilisateurs du cluster source et k le nombre de candidats, est de l'ordre de $O(M \times s \times k)$, où M est le nombre d'itérations de projection. La phase de réaffectation discrète peut, dans le pire cas, devenir combinatoire, mais en pratique ce coût reste maîtrisé car on limite la taille des clusters sources traités et le nombre de candidats voisins.

4 Résultats obtenus

4.1 Algorithme p-center optimisé

4.1.1 Résultats chiffrés

L'algorithme obtient une solution faisable pour tous les scénarios testés : aucun cluster invalide n'est obtenu, ce qui confirme le respect des contraintes de rayon et de capacité.

Demande	Capacité	Clusters	Invalides	Rayon moyen	Remplissage moyen	Temps
CIR	1 Gbps	13 413	0	31,8 km	8,0 %	87,8 s
CIR	2 Gbps	13 407	0	31,8 km	4,0 %	88,9 s
CIR	4 Gbps	13 412	0	31,8 km	2,0 %	90,4 s
PIR	1 Gbps	20 518	0	24,9 km	60,5 %	53,2 s
PIR	2 Gbps	14 788	0	30,8 km	42,0 %	71,2 s
PIR	4 Gbps	13 466	0	31,9 km	23,0 %	85,4 s

TABLE 1 – Résultats obtenus avec l'algorithme p-center capacitaire optimisé.

Les résultats montrent que le choix de la demande influence fortement le nombre de clusters. Avec le CIR, le nombre de clusters reste presque constant autour de 13 400, ce qui indique que la contrainte géographique domine. Avec le PIR, la capacité devient plus contraignante, surtout à 1 Gbps où 20 518 clusters sont nécessaires. À 4 Gbps, les résultats CIR et PIR deviennent presque identiques, ce qui confirme que la capacité n'est alors plus le facteur limitant principal.

Des graphiques complémentaires propres à l'algorithme p-center optimisé sont présentés en annexe. Ils permettent de visualiser les écarts entre CIR et PIR, les distributions de rayons et de remplissage, ainsi que la répartition géographique des terminaux et des centres de clusters.

4.1.2 Complexité

Côté complexité, la construction du KD-tree coûte $O(N \log N)$, puis les recherches locales et les validations géométriques donnent une borne pessimiste de l'ordre de $O(N^2 \log N)$ dans le pire cas.

En pratique, les temps mesurés restent de l'ordre d'une à deux minutes par scénario grâce aux micro-zones, au KD-tree et au nombre borné de candidats testés. Ces temps sont satisfaisants pour une planification hors-ligne, mais restent trop élevés pour une utilisation en temps réel.

4.2 Algorithme Gravity Mean-Shift

4.2.1 Résultats chiffrés

L'algorithme Gravity Mean-Shift a été évalué sur les six scénarios combinant les deux modes (CIR et PIR) et les trois paliers de capacité. Dans chaque cas, la phase gravitaire est suivie du post-traitement de pruning et de fusion décrit dans la section post-traitement. Aucun cluster invalide n'est produit, confirmant le respect simultané des contraintes de distance et de capacité.

Mode	Capacité	Terminaux	Clusters (avant)	Clusters (après post)	Remplissage moyen	Temps total
CIR	1 Gbps	87 989	18 671	15 895	6,72%	5,9 s
CIR	2 Gbps	87 989	18 670	15 890	3,36%	5,9 s
CIR	4 Gbps	87 989	18 670	15 890	1,68%	5,9 s
PIR	1 Gbps	88 325	25 473	22 656	54,77%	7,3 s
PIR	2 Gbps	87 989	19 950	17 082	36,32%	6,0 s
PIR	4 Gbps	87 989	18 650	15 914	19,49%	6,3 s

TABLE 2 – Résultats obtenus avec l'algorithme Gravity Mean-Shift et son post-traitement.

On observe deux comportements distincts selon le mode utilisé. En mode CIR, le nombre de clusters est pratiquement le même quelle que soit la capacité ($\approx 18\,670$ avant post-traitement), et le remplissage moyen est très faible, entre 1,68 et 6,72 %. Cela s'explique par le fait que les débits CIR sont naturellement petits : la contrainte géographique des 45 km est atteinte bien avant la limite de capacité, et augmenter le plafond en Gbps ne change donc presque rien au résultat final.

En mode PIR, la capacité joue un rôle bien plus important. À 1 Gbps, certains terminaux ont une demande individuelle qui dépasse le plafond et doivent être découpés en terminaux virtuels (88 325 au lieu de 87 989), ce qui explique le nombre de clusters plus élevé (22 656) et le remplissage moyen de 54,77 %. À 2 et 4 Gbps, les résultats se rapprochent progressivement de ceux obtenus en CIR, la contrainte de capacité devenant de moins en moins bloquante.

Dans tous les scénarios, le post-traitement réduit le nombre de clusters de 15 à 20 % sans jamais violer les contraintes.

4.2.2 Complexité

La complexité de l'algorithme, analysée en détail lors de la présentation de l'algorithme, est de l'ordre de $O(N \log N)$ en moyenne, dominée par le tri initial et les requêtes au K-D Tree. Les temps d'exécution mesurés (5 à 7 secondes pour 90 000 terminaux) confirment cette analyse : le déplacement gravitaire est borné à 3 itérations par satellite et opère sur un sous-ensemble local $k \ll N$, si bien que le coût total reste linéarisé par l'arbre.

Cette rapidité d'exécution est un atout important dans notre contexte. Les satellites LEO effectuent un passage visible au-dessus d'une zone en une dizaine de minutes seulement : la carte des utilisateurs actifs et leurs demandes en débit évoluent donc en permanence. Un algorithme en $O(N^2)$ ou $O(N^3)$ ne pourrait pas suivre ce rythme sans infrastructure de calcul dédiée. Ici, recalculer entièrement le plan de faisceaux prend quelques secondes, ce qui laisse envisager une mise à jour dynamique des clusters entre deux passages orbitaux, voire directement depuis la station de contrôle au sol sans délai perceptible pour les utilisateurs.

4.3 Algorithme split-merge avec réduction de Boyle-Dykstra

4.3.1 Résultats chiffrés

Cette approche produit une solution faisable dans les six scénarios testés : aucun cluster invalide n'est observé, ce qui confirme le respect simultané de la contrainte géographique de 90 km et la contrainte de capacité. Le tableau distingue les exécutions où la capacité est évaluée sur le *CIR* de celles où elle est évaluée sur le *PIR*.

Débit	Capacité	Terminaux (virtuels inclus)	Clusters	Invalides	Diamètre moyen	Remplissage moyen	Temps
CIR	1 Gbps	87 989	13 852	0	66,2 km	7,5 %	202 s
CIR	2 Gbps	87 989	13 851	0	66,2 km	3,8 %	204 s
CIR	4 Gbps	87 989	13 851	0	66,2 km	1,9 %	200 s
PIR	1 Gbps	88 325	20 415	0	49,0 km	59,4 %	369 s
PIR	2 Gbps	87 989	14 745	0	64,3 km	41,1 %	229 s
PIR	4 Gbps	87 989	13 892	0	66,2 km	21,8 %	199 s

TABLE 3 – Résultats obtenus avec l'algorithme split-merge suivi d'une réduction de Boyle-Dykstra.

Deux comportements apparaissent nettement. Lorsque la contrainte porte sur le CIR, le nombre de clusters varie très peu entre 1, 2 et 4 Gbps. Cela signifie que, dans ce cas, la contrainte de rayon devient dominante et limite davantage la solution que la capacité.

Lorsque la contrainte porte sur le PIR, le cas 1 Gbps est plus difficile. Une partie des utilisateurs dépasse alors la capacité autorisée et doit être scindée en terminaux virtuels supplémentaires. On obtient donc davantage de terminaux à traiter, davantage de clusters, et un temps d'exécution plus élevé.

Le remplissage moyen confirme cette lecture. Sous contrainte PIR, les clusters utilisent une part importante de la capacité disponible, surtout à 1 Gbps. Sous contrainte CIR, au contraire, la charge moyenne par cluster reste faible. La géométrie structure donc l'essentiel de la solution finale.

4.3.2 Complexité

Complexité algorithmique

Soit N le nombre de terminaux effectivement manipulés, après éventuelle virtualisation. Dans le pire cas, la phase de division et la phase de fusion peuvent être majorées par un coût de l'ordre de $O(N^2)$, car elles reposent sur de nombreuses validations géométriques et sur des recherches locales répétées.

Si l'on note C le nombre de clusters actifs, k le nombre de clusters candidats testés par Boyle-Dykstra, s la taille maximale d'un cluster source traité, et P le nombre d'itérations de projection, alors ce post-traitement peut être borné par $O(CksP)$ lorsque ces paramètres sont fixés.

Ainsi, d'un point de vue asymptotique, le pire cas reste dominé par les phases de division et de fusion en $O(N^2)$. En revanche, dans nos expériences, la phase Boyle-Dykstra est souvent la plus coûteuse en temps d'exécution, car ses projections et ses réaffectations ont une constante de coût élevée.

4.4 Comparaison globale des approches

Les trois algorithmes proposés produisent des solutions faisables : aucun cluster invalide n'est observé dans les scénarios testés. La comparaison porte donc principalement sur deux critères : le nombre de clusters obtenus et le temps d'exécution.

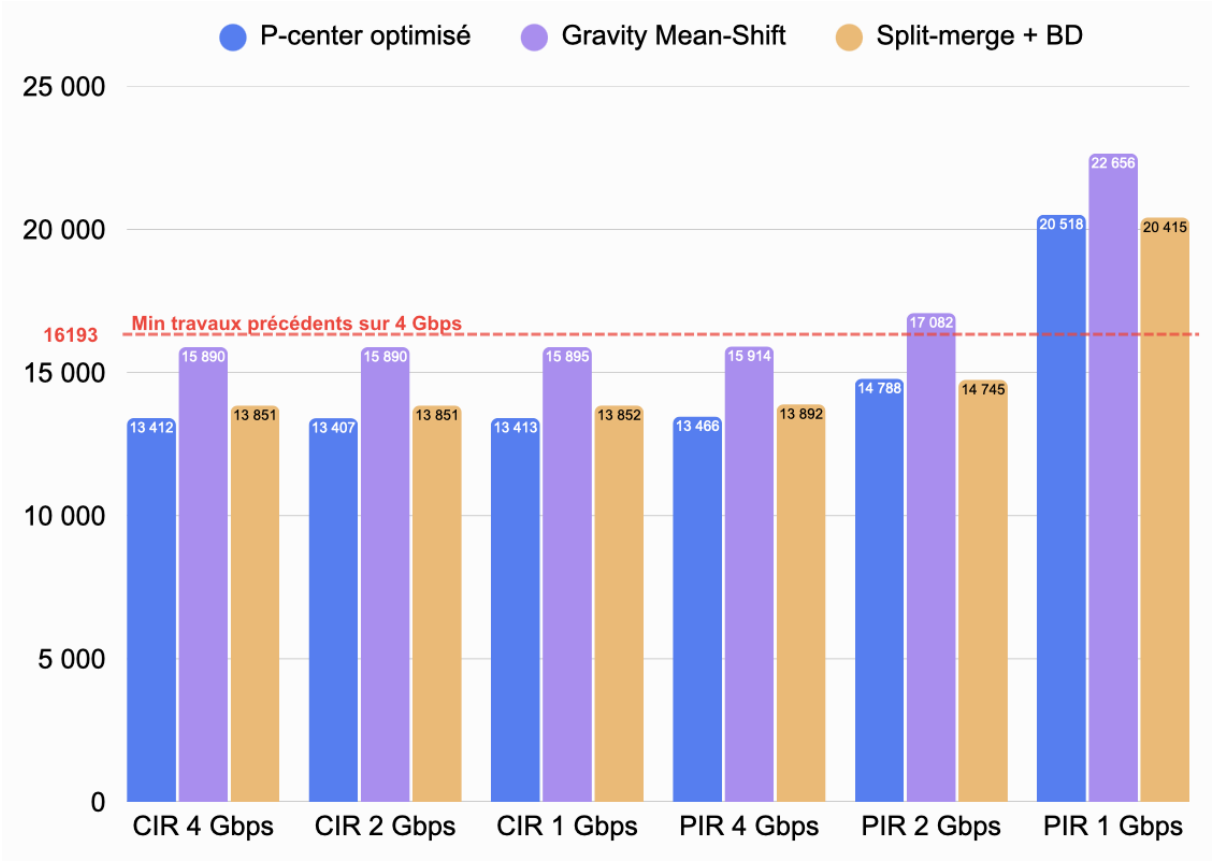


FIGURE 2 – Comparaison du nombre de clusters obtenus par les trois algorithmes selon le scénario étudié.

La figure 2 montre que le p-center optimisé et l'approche split-merge avec Boyle-Dykstra donnent les meilleurs résultats en nombre de clusters. En CIR, le p-center optimisé descend jusqu'à 13 407 clusters, contre 13 851 pour split-merge et environ 15 890 pour Gravity Mean-Shift. En PIR, split-merge est légèrement meilleur sur les scénarios 1 et 2 Gbps, mais au prix d'un temps de calcul nettement plus élevé.

La ligne de référence correspond au meilleur résultat communiqué dans les travaux précédents, soit 16 193 clusters après post-traitement pour le scénario à 4 Gbps. Faute de détails supplémentaires, elle est utilisée comme repère indicatif plutôt que comme comparaison complète scénario par scénario. Nos approches passent sous cette référence dans plusieurs cas comparables, avec un gain maximal d'environ 17 %.

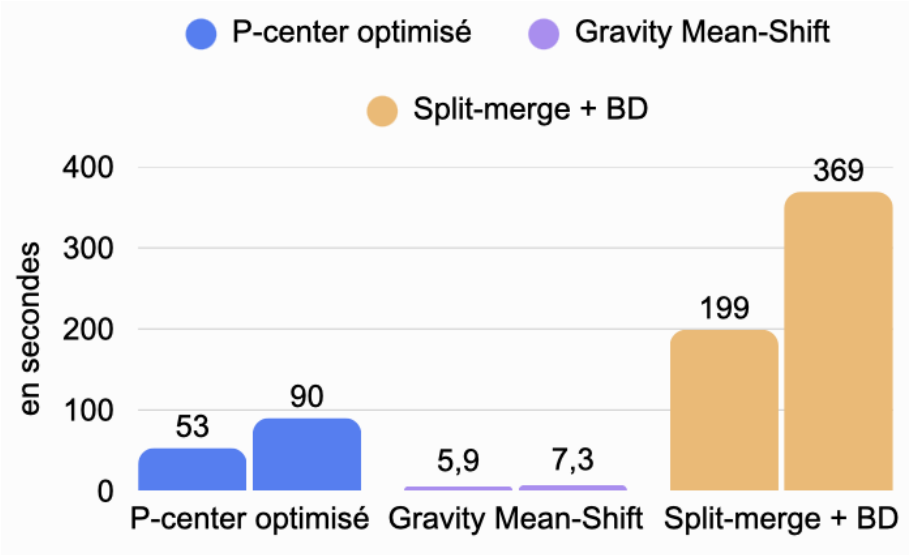


FIGURE 3 – Comparaison des temps d'exécution des trois algorithmes.

La figure 3 met en évidence un compromis entre la qualité de la solution et le temps de calcul. Gravity Mean-Shift est l'approche la plus rapide, avec des temps de l'ordre de quelques secondes, mais elle génère davantage de clusters. À l'inverse, split-merge avec Boyle-Dykstra produit de très bons résultats, mais avec des temps pouvant dépasser plusieurs minutes. Le p-center optimisé constitue donc le meilleur compromis global : il obtient un faible nombre de clusters tout en conservant un temps d'exécution raisonnable pour une planification hors-ligne.

5 Conclusion

Ce projet visait à regrouper près de 90 000 terminaux en un nombre minimal de clusters, sous deux contraintes : un rayon maximal de 45 km et une capacité de 1, 2 ou 4 Gbps par cluster. Nous avons comparé plusieurs heuristiques en faisant varier l'ordre de traitement des terminaux et l'ordre d'agrégation des points.

Les trois approches produisent des solutions faisables, sans cluster invalide. Les résultats montrent que le choix CIR/PIR est déterminant : en CIR, la contrainte géographique domine, tandis qu'en PIR la capacité devient plus contraignante, surtout à 1 Gbps. Nos meilleurs résultats atteignent environ 13 400 clusters et passent sous la meilleure référence disponible dans les travaux précédents.

Minimiser le nombre de clusters est pertinent pour réduire le nombre de faisceaux et les temps d'attente en *beam-hopping*. Cependant, ce critère ne suffit pas seul : il peut conduire à des faisceaux très chargés, mal équilibrés ou à un risque de point de défaillance unique. Le p-center optimisé apparaît comme le meilleur compromis global entre nombre de clusters, temps de calcul et respect des contraintes. Des améliorations futures pourraient intégrer la résilience, l'équilibrage de charge, la QoS et une couverture redondante des terminaux critiques.

A Annexes

A.1 Graphiques complémentaires de l'algorithme p-center optimisé

Les figures de cette annexe concernent uniquement l'algorithme p-center capacitaire optimisé présenté dans la section correspondante. Elles ne comparent pas l'ensemble des algorithmes du rapport, mais les résultats obtenus avec cet algorithme selon deux métriques de demande : le CIR et le PIR.

A.1.1 Comparaison globale entre CIR et PIR

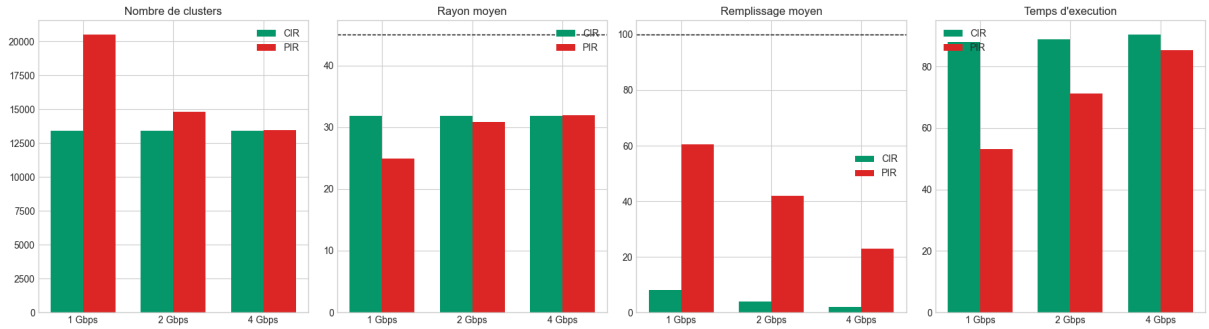


FIGURE 4 – Résultats de l'algorithme p-center optimisé selon la demande utilisée : CIR ou PIR.

La figure 4 montre que l'écart entre CIR et PIR est surtout visible pour les faibles capacités. À 1 Gbps, le PIR impose beaucoup plus de clusters, car la capacité devient réellement contraignante. À 4 Gbps, le nombre de clusters devient presque identique, ce qui indique que la contrainte géographique domine.

A.1.2 Distribution des rayons et des taux de remplissage

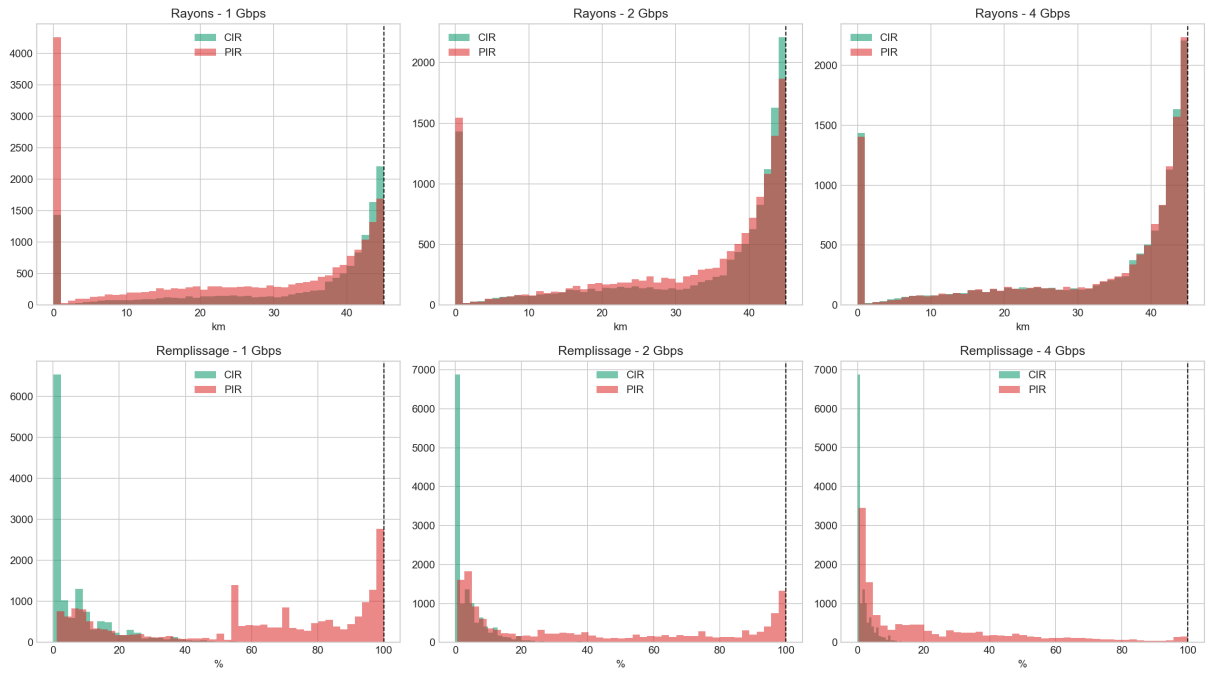


FIGURE 5 – Distribution des rayons et des taux de remplissage des clusters obtenus avec l'algorithme p-center optimisé.

La figure 5 confirme que les clusters produits respectent la limite de 45 km. Les clusters PIR sont globalement mieux remplis que les clusters CIR, car le PIR représente une demande plus élevée. En CIR, le faible remplissage montre que le nombre de clusters est surtout imposé par la répartition géographique des terminaux.

A.1.3 Répartition géographique des terminaux et des centres

La carte suivante concerne uniquement l'algorithme p-center optimisé. Elle compare, pour chaque capacité, la densité géographique des terminaux avec celle des centres de clusters obtenus en CIR et en PIR. Les couleurs plus foncées correspondent aux zones contenant davantage de terminaux ou de centres. L'échelle de couleur est logarithmique afin de rendre lisibles les zones très denses.

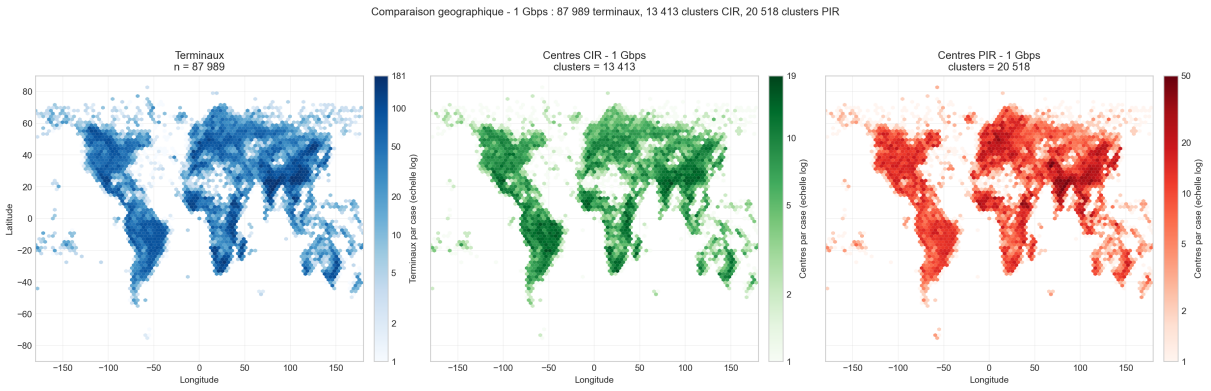


FIGURE 6 – Répartition géographique des terminaux et des centres CIR/PIR obtenus avec l'algorithme p-center optimisé pour le scénario 1 Gbps.

Pour 1 Gbps, le scénario PIR nécessite nettement plus de centres que le scénario CIR, en particulier dans les zones denses où la capacité devient contraignante.

Références

[1] H. Shehata, H. Inaltekin, and I. B. Collings, "Ground user clustering for adaptive multibeam geo satellite networks.," *Sensors (Basel, Switzerland)*, vol. 26, no. 8, 2026.

[2] T. V. Chien, N. M. Quan, T. N. Do, C. Le, T. N. Nguyen, and S. Chatzinotas, "Fast beam placement for ultra-dense leo networks," *IEEE Wireless Communications Letters*, vol. 14, no. 3, pp. 621–625, 2025.

[3] J. Tang, D. Bian, G. Li, J. Hu, and J. Cheng, "Optimization method of dynamic beam position for LEO beam-hopping satellite communication systems," *IEEE Access*, vol. 9, pp. 57578–57588, 2021.

[4] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0 : Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[5] OpenDSA Project, "KD Trees - CS3 Data Structures & Algorithms." https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/KDtree.html, 2026. Consulté le 21/05/2026.

[6] D. Comaniciu and P. Meer, "Mean shift : A robust approach toward feature space analysis," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 5, pp. 603–619, 2002.

[7] R. L. Dykstra, "An algorithm for restricted least squares regression," *Journal of the American Statistical Association*, vol. 78, no. 384, pp. 837–842, 1983.